

24 Août 2026

Fondations du Machine Learning & La Régression

Patrick Randriamanambola, Data Engineer

LE MACHINE LEARNING

Définitions, concepts fondamentaux et catégorisation des
algorithmes.

Les Catégories



Apprentissage Supervisé

Apprentissage via des données étiquetées pour prédire des cibles spécifiques.



Apprentissage Non Supervisé

Exploration autonome de données brutes pour découvrir des structures.



Apprentissage Semi-Supervisé

Combinaison d'un petit volume étiqueté et d'un grand volume brut.

L'Apprentissage Supervisé

L'algorithme apprend à partir d'exemples corrects fournis par
l'humain.

L'Apprentissage Supervisé



Données étiquetées

Utilisation d'ensembles de données contenant des entrées et sorties connues.



Entraînement guidé

Les paires d'exemples permettent d'entraîner et « superviser » l'algorithme.



Mesure de précision

Évaluation de l'écart entre la prédiction et l'étiquette réelle connue.

L'Apprentissage Supervisé

La Régression

Comprendre la relation entre variables pour prédire des valeurs numériques continues.

Exemple :

Prédiction de Prix de l'immobilier.

La Classification

Assigner avec précision des données dans des catégories prédéfinies.

Exemple :

Détection de Spam ou Non Spam.

L'Apprentissage Non Supervisé

Découverte autonome de structures dans des données non
étiquetées.

L'Apprentissage Non Supervisé



Pas d'étiquettes

Utilise des algorithmes pour analyser et regrouper des données brutes.



Sans intervention

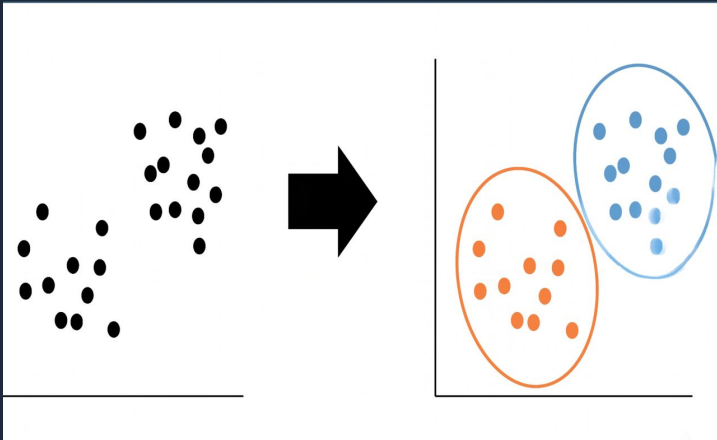
Révèle la structure inhérente des données sans supervision humaine.



Découverte autonome

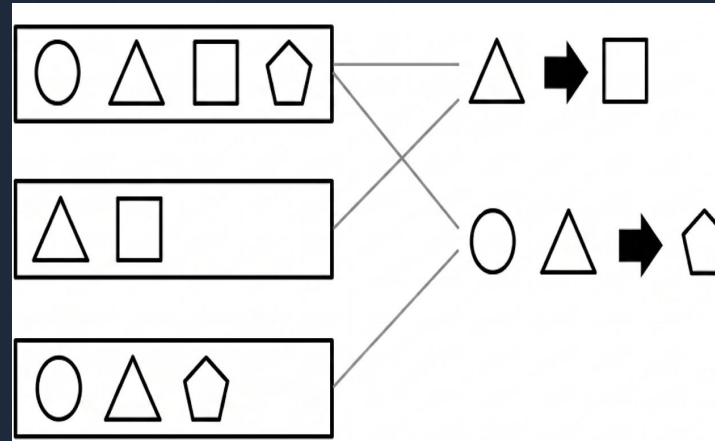
Utilise des algorithmes pour analyser et regrouper des données brutes.

L'Apprentissage Non Supervisé



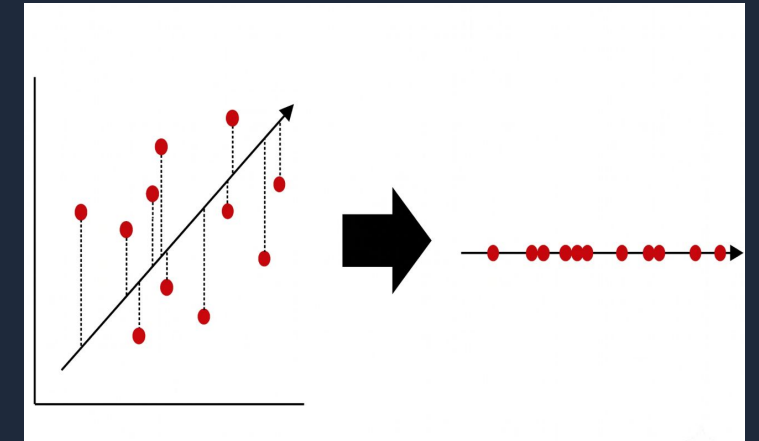
Le Clustering

Regrouper des données selon leurs similitudes (Ex: Segmentation de marché).



L'Association

Déterminer les relations entre variables (Ex: Système de recommandation).



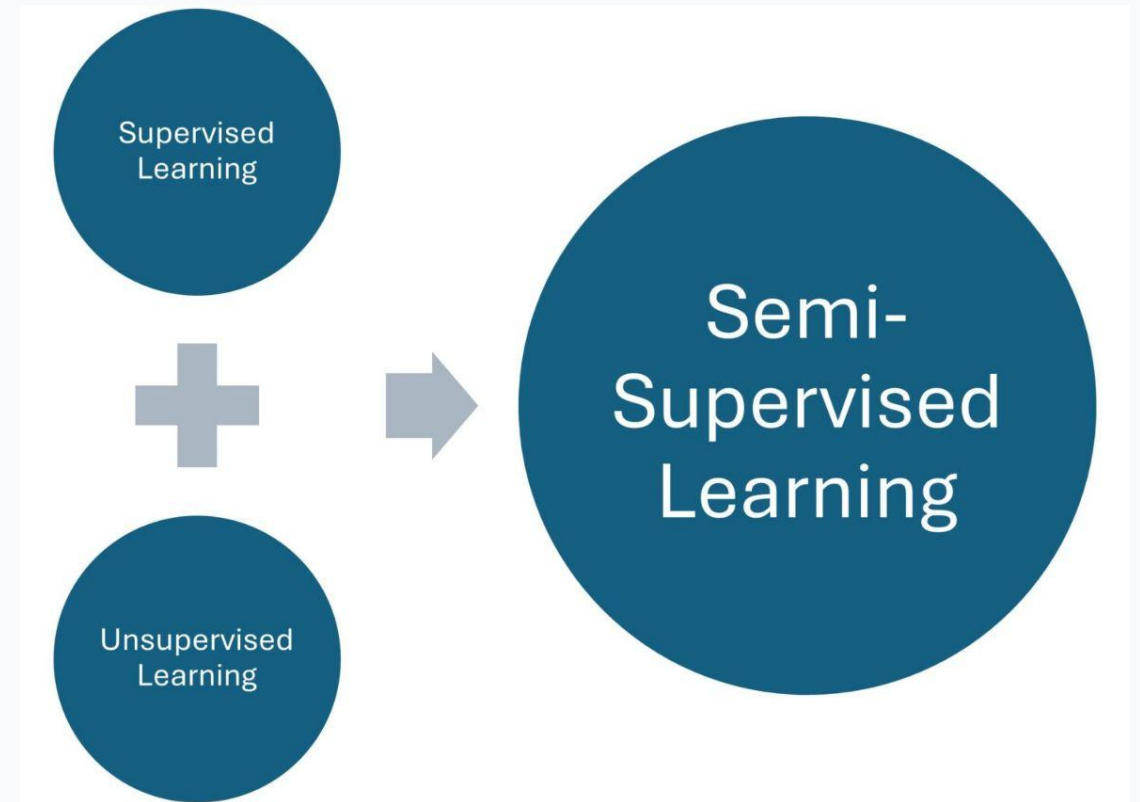
La Réduction

Réduire la dimension des données complexes (Ex: Compression PCA).

L'Apprentissage Semi-Supervisé

- Combine une petite quantité de données étiquetées avec un volume massif de données non étiquetées.
- Solution idéale lorsque l'étiquetage humain complet est impossible ou trop coûteux.

Cas d'usage : L'imagerie médicale pour prédire des pathologies à partir de quelques scanners annotés.



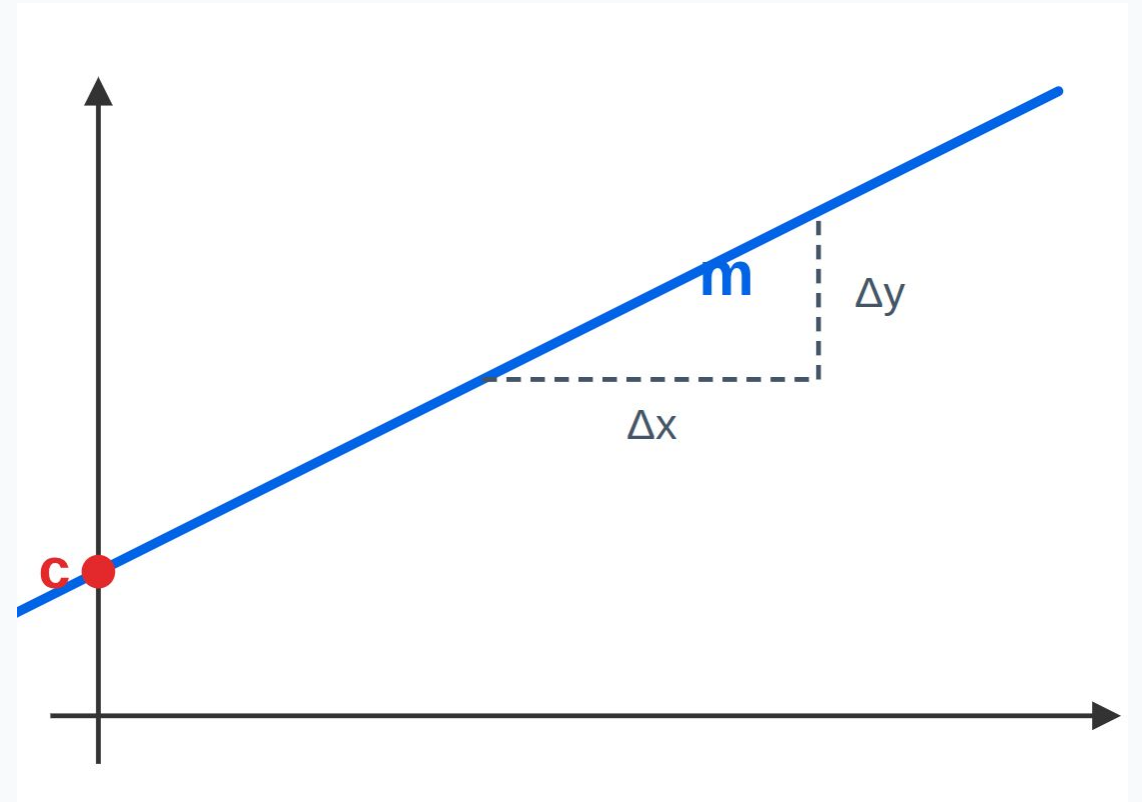
LA RÉGRESSION

Un outil extrêmement puissant pour modéliser les relations entre les variables.

Les composants de l'équation

$$y = m \cdot x + c$$

- **y** : Variable dépendante à prédire.
- **x** : Variable indépendante (facteur d'entrée).
- **m** : Pente de la droite de régression.
- **c** : Ordonnée à l'origine (y - intercept).



Les Données



L'importance de la préparation

Note : La qualité (nettoyage) et la quantité de ces données initiales sont absolument déterminantes pour la précision du futur modèle.

Des données corrompues entraîneront des prédictions erronées (Garbage In, Garbage Out).

LA RÉGRESSION LINÉAIRE SIMPLE

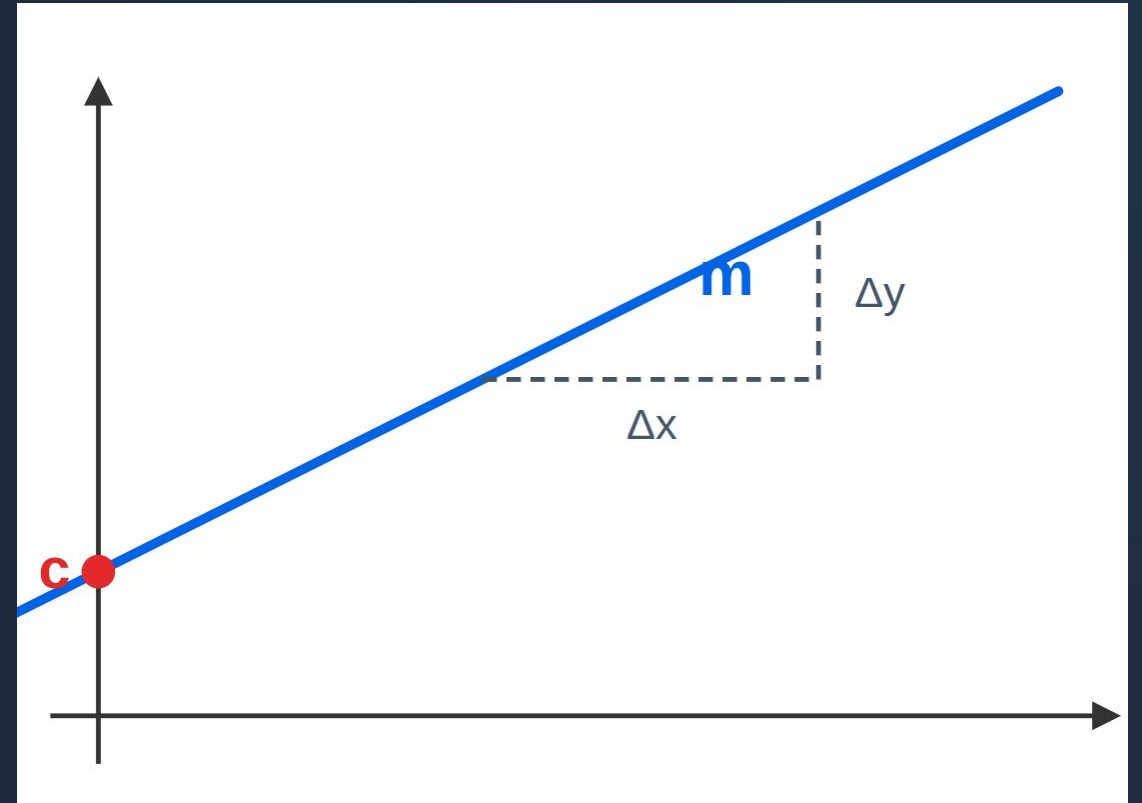
**Modélisation mathématique sur une seule
dimension.**

Le Modèle Mathématique

On pose l'hypothèse d'une relation linéaire tel que :

$$y = m \cdot x + c$$

Notre but est de trouver la meilleure pente (m) et l'ordonnée à l'origine (c).



Estimation des paramètres (MCO)

L'estimateur des Moindres Carrés Ordinaires (MCO) utilise les statistiques descriptives :

- **La Variance de X** : Mesure la dispersion des données d'entrée.
- **La moyenne de y** : Point central de notre variable cible.
- **La moyenne de X** : Point central de notre variable explicative.
- **Objectif** : La droite passe toujours par le point (moyenne X, moyenne Y).

Implémentation avec Python

1. Calcul Algébrique Numpy

En utilisant NumPy, nous pouvons calculer directement la pente m et l'ordonnée c de manière vectorisée.

```
import numpy as np

# Calcul des moyennes de X et y
mean_x = np.mean(x)
mean_y = np.mean(y)

# Calcul des coefficients
m = np.sum((x - mean_x) * (y - mean_y)) /
    np.sum((x - mean_x)**2)
c = mean_y - m * mean_x
```

LA RÉGRESSION LINÉAIRE MULTIPLE

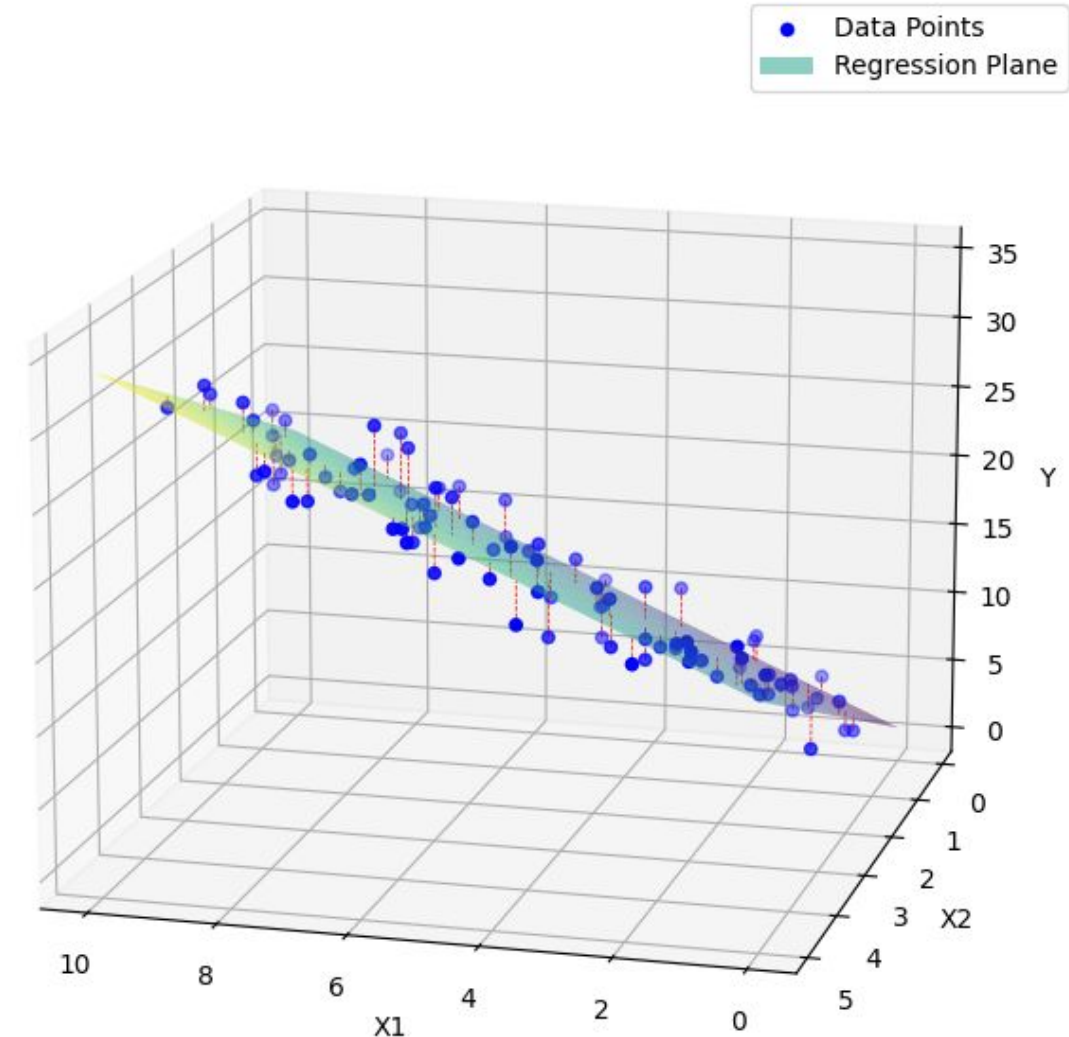
Modélisation avec de multiples caractéristiques et dimensions.

Le Modèle Mathématique

Tout comme avec la régression linéaire simple, on pose l'hypothèse d'une relation linéaire tel que :

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

Où x_n représentent les variables indépendantes multiples.



L'Écriture Matricielle

$$y = X \cdot \beta$$

Notre but est de trouver l'estimateur β , qui est le vecteur des poids de chaque variable indépendante.

- **β (Beta)** : Vecteur contenant l'ordonnée à l'origine et tous les coefficients de pente.
- **X** : Matrice complète des caractéristiques d'entrée (Features).

Estimation des paramètres (Matrice)

Équation Normale

On ajoute une colonne de "1" à X pour intégrer l'ordonnée à l'origine, puis on résout l'équation par inversion matricielle.

```
import numpy as np

ones = np.ones(shape=(len(X), 1))
X = np.append(ones, X, axis=1)

# Calcul de l'inverse de la matrice X.T.dot(X)
inv = np.linalg.inv(X.T.dot(X))

# Calcul des coefficients
beta = inv.dot(X.T).dot(y)
```

L'évaluation de la performance

Pour valider l'efficacité du modèle multiple, on évalue ses performances avec le coefficient de détermination, également appelé score **R²**.

Plus cette valeur se rapproche de 1, plus notre modèle s'ajuste parfaitement aux données. À 0, le modèle ne fait pas mieux que la moyenne.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

La limite du MCO

La Solution !?

Le goulot d'étranglement calculatoire

L'équation offre la possibilité d'avoir une solution directe. Toutefois, l'inversion de la matrice devient trop lente et inefficace avec des ensembles de données massifs.

L'approche algébrique est computationnellement lourde. Calculer l'inverse de $X^T X$ est un gouffre de mémoire.

LA DESCENTE DE GRADIENT

L'algorithme d'optimisation au cœur du Machine Learning
moderne.

Pourquoi la descente de Gradient ?

- ✓ **Efficacité Mémoire** : Contourne le besoin critique d'inverser d'immenses matrices.
- ✓ **Scalabilité** : Idéal pour le Big Data, peut traiter les données par lots (Mini-batch).
- ✓ **Universalité** : C'est le moteur mathématique utilisé pour entraîner les réseaux de neurones complexes.

Le principe de la descente de Gradient



Itératif

Algorithme d'optimisation :
Minimise une fonction cible en
ajustant pas à pas les paramètres.



Minimisation

Quantifie et réduit l'écart absolu
entre les prédictions et les valeurs
réelles.



La Pente

Atteint le minimum en se
déplaçant dans le sens opposé du
gradient de la fonction.

Les étapes de la Descente de Gradient

Fonction de coût, calcul du gradient et
itérations.

La fonction de Coût

L'Erreur Quadratique

La condition d'exécutabilité nécessite impérativement que la fonction objectif soit entièrement dérivable.

```
import numpy as np

# La fonction de coût
def cost(X, y, theta):
    m = len(y)
    predictions = X.dot(theta)
    cost = (1/m) *
    np.sum(np.square(predictions-y))
    return cost
```

La Descente (Formule)

$$\theta_t = \theta_{t-1} - \alpha \nabla J\theta_{t-1}$$

- **Valeur de θ (t)** : Nouveau paramètre mis à jour.
- **Valeur de θ (t-1)** : Valeur du paramètre à l'itération précédente.
- **α (Alpha)** : Pas d'apprentissage (Learning Rate).
- **∇J** : Variation de la fonction coût vis-à-vis du changement de θ (Gradient).

La Descente: Python

Boucle d'optimisation

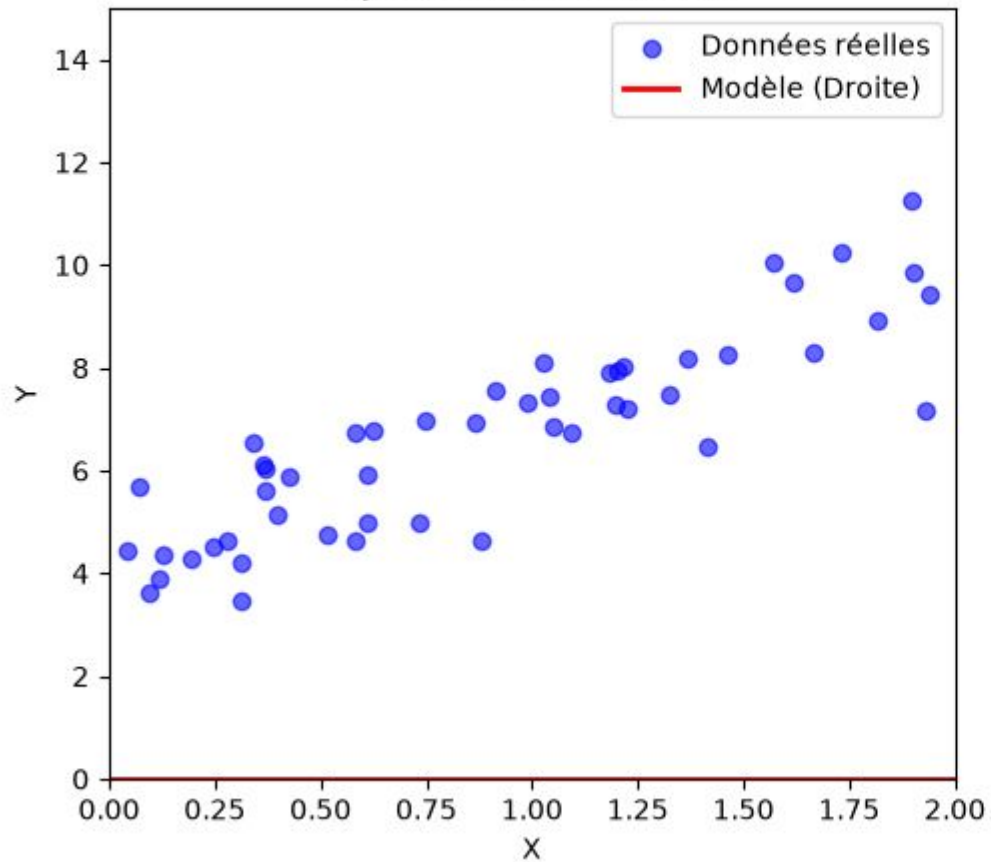
Le code met à jour itérativement les poids en utilisant la dérivée de l'erreur.

```
def gradient_descent(X, y, theta, alpha,
iterations):
    m = len(y)
    cost_hist = np.zeros(iterations)
    theta_hist = np.zeros((iterations,2))
    for i in range(iterations):
        pred = np.dot(X,theta)
        theta = theta - (1/m)*alpha*
(X.T.dot((pred-y)))
        theta_hist[i,:] = theta.T
        cost_hist[i] = cost(X,y,theta)
    return theta, cost_hist, theta_hist
```

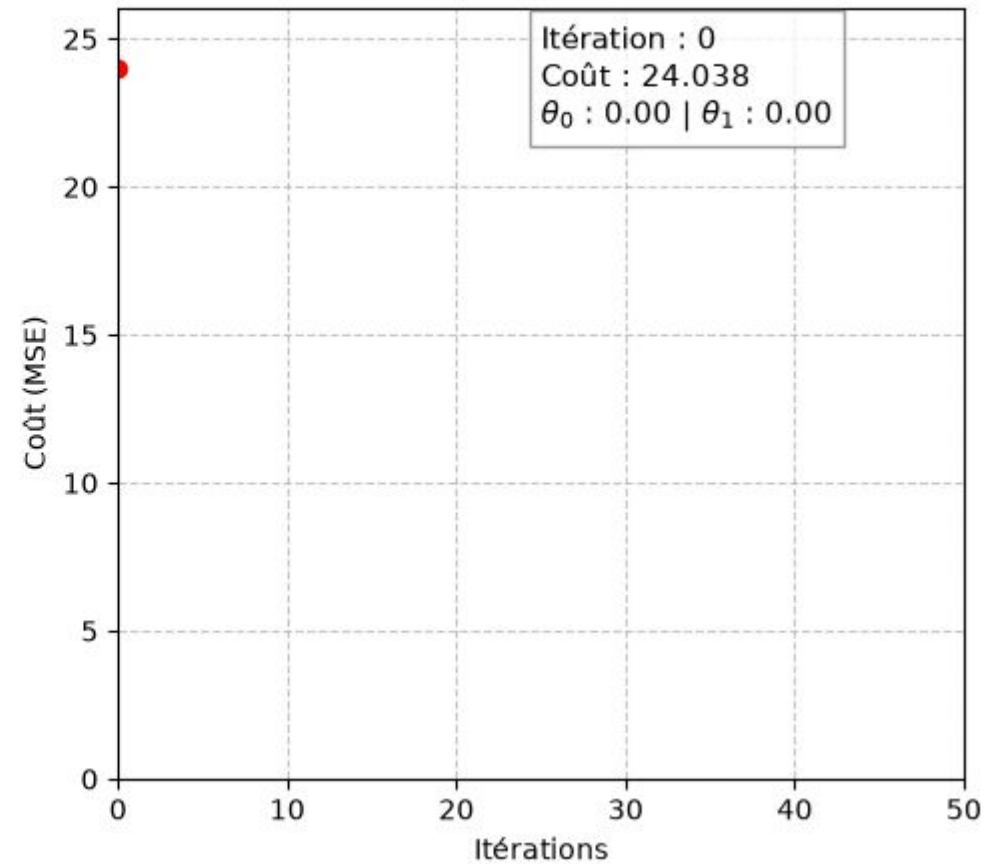
La Descente: Python

Descente de Gradient : Régression Linéaire

Ajustement du Modèle



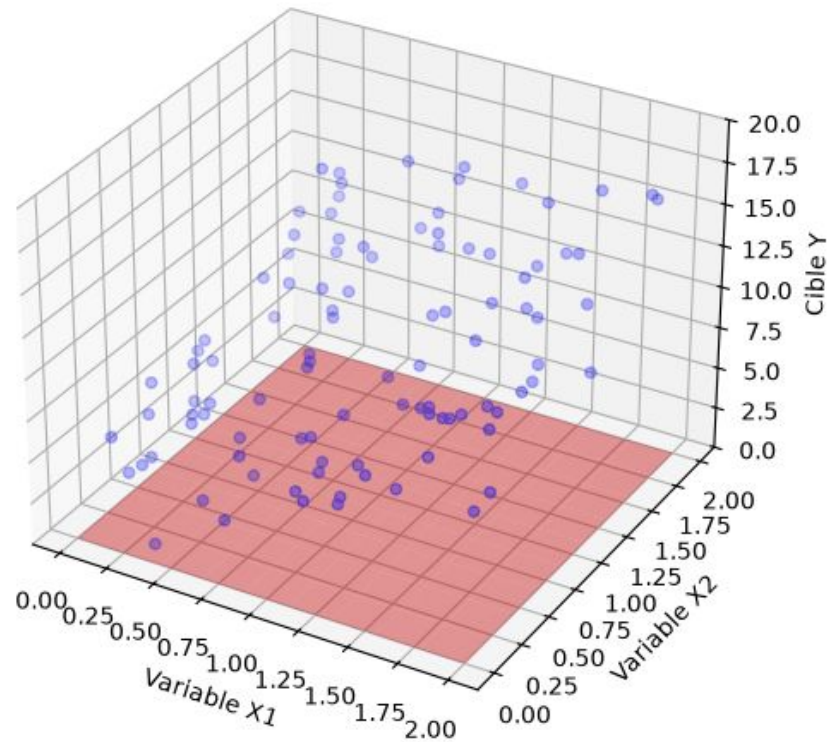
Fonction de Coût (Erreur)



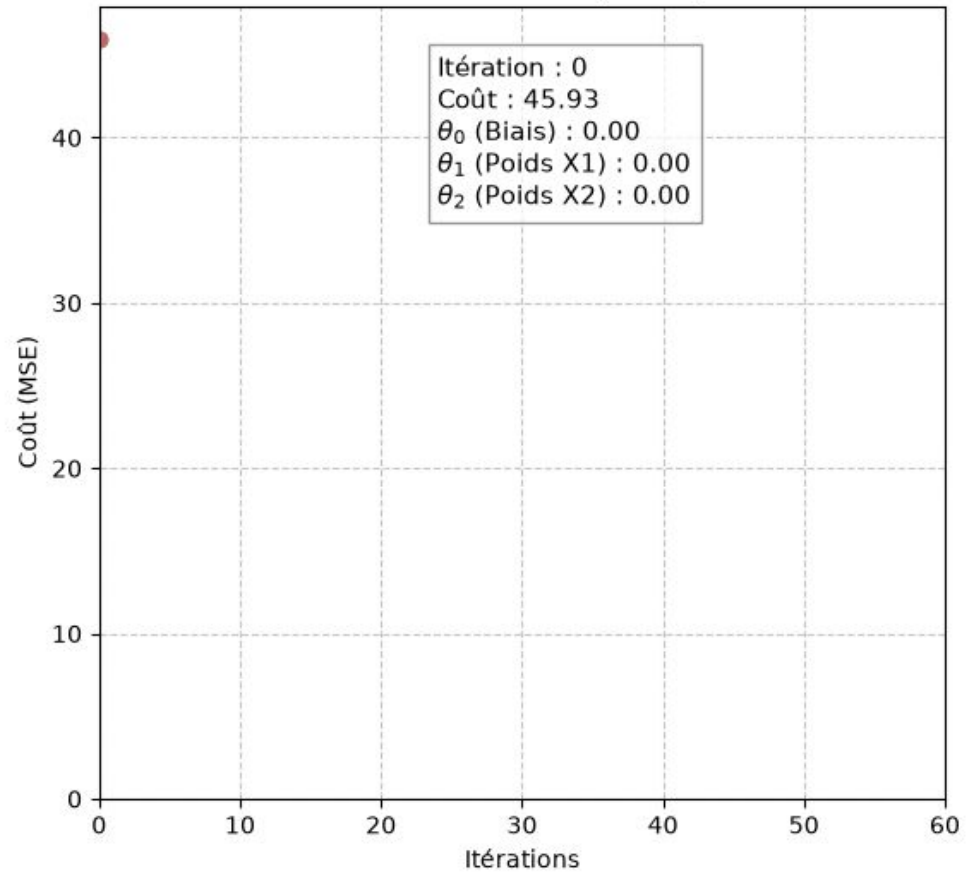
La Descente: Python

Descente de Gradient : Régression Linéaire Multiple

Ajustement du Plan de Régression



Fonction de Coût (Erreur)



Validité de la Régression Linéaire

**Les théorèmes et hypothèses mathématiques sous-jacentes
(Gauss-Markov).**

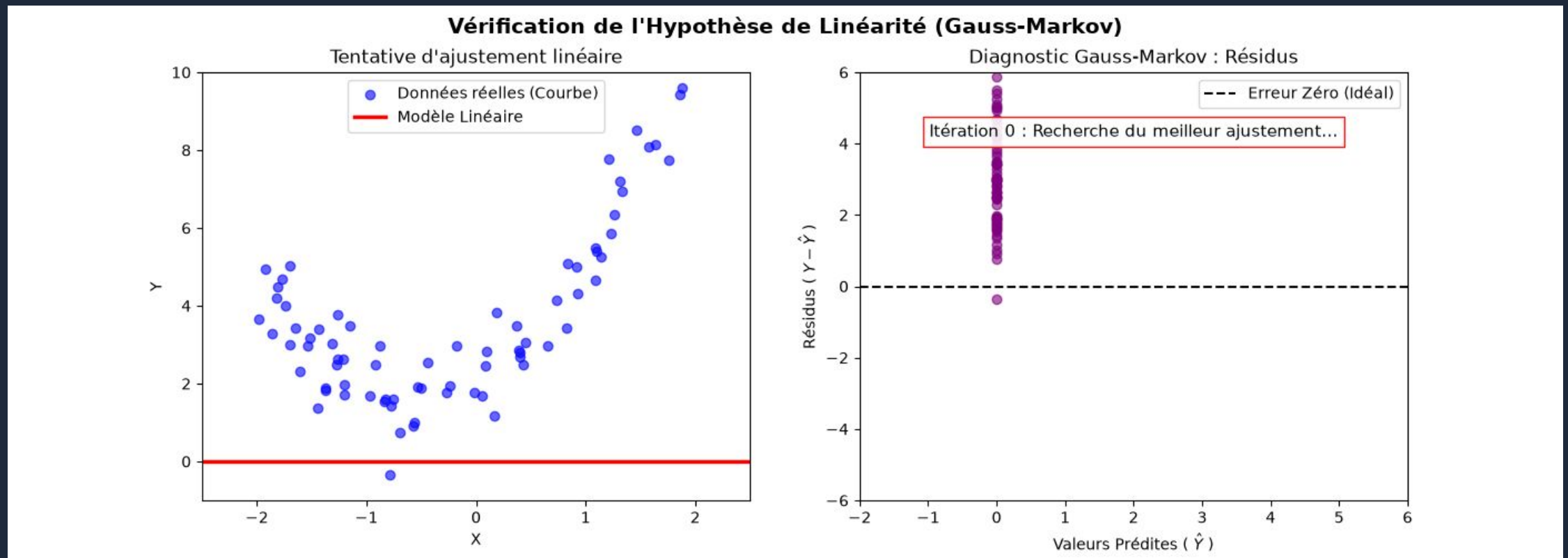
Hypothèses de Gauss-Markov

Pour que les estimateurs MCO soient BLUE (Best Linear Unbiased Estimators) :

- ✓ **Condition 1 : Linéarité** : La relation dans les paramètres doit être linéaire.
- ✓ **Condition 2 : Homoscédasticité** : La variance des résidus doit être constante.
- ✓ **Condition 3 : Indépendance** : Les erreurs ne doivent pas être autocorrélées.
- ✓ **Condition 4 : Non-colinéarité** : Les variables d'entrée ne doivent pas être parfaitement corrélées entre elles.

Condition 1 : La Linéarité

Si la vraie relation est fortement courbée (exponentielle), ajuster une droite stricte produira de très mauvaises prédictions.

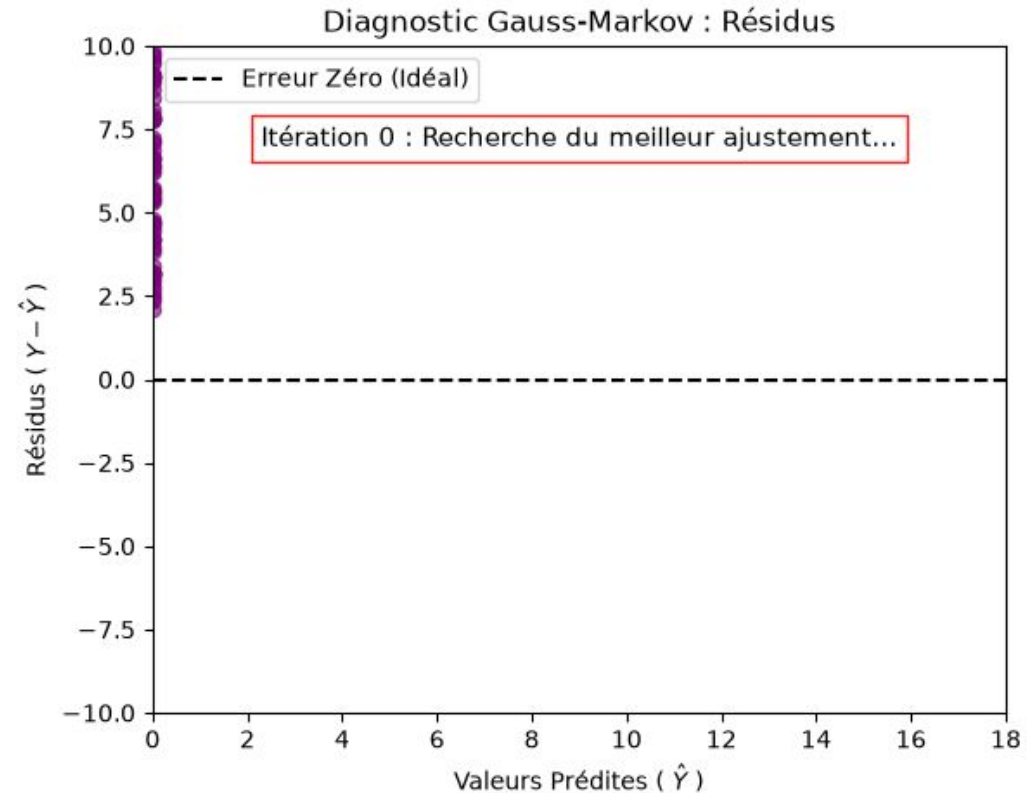
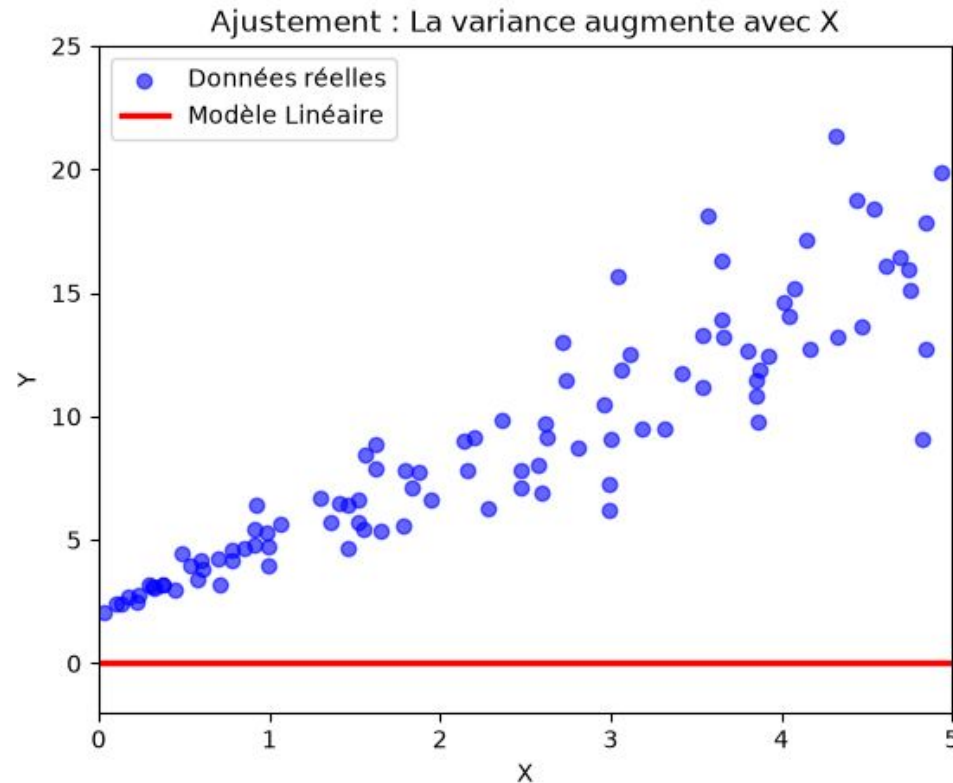


Solution : Effectuer des transformations logarithmiques ou polynomiales sur les caractéristiques d'entrée.

Condition 2 : Homoscédasticité

Exige que la variance des termes d'erreur soit constante. Si l'écartement des erreurs grandit proportionnellement à X , on parle d'**hétéroscédasticité**.

Vérification de l'Homoscédasticité (Gauss-Markov)

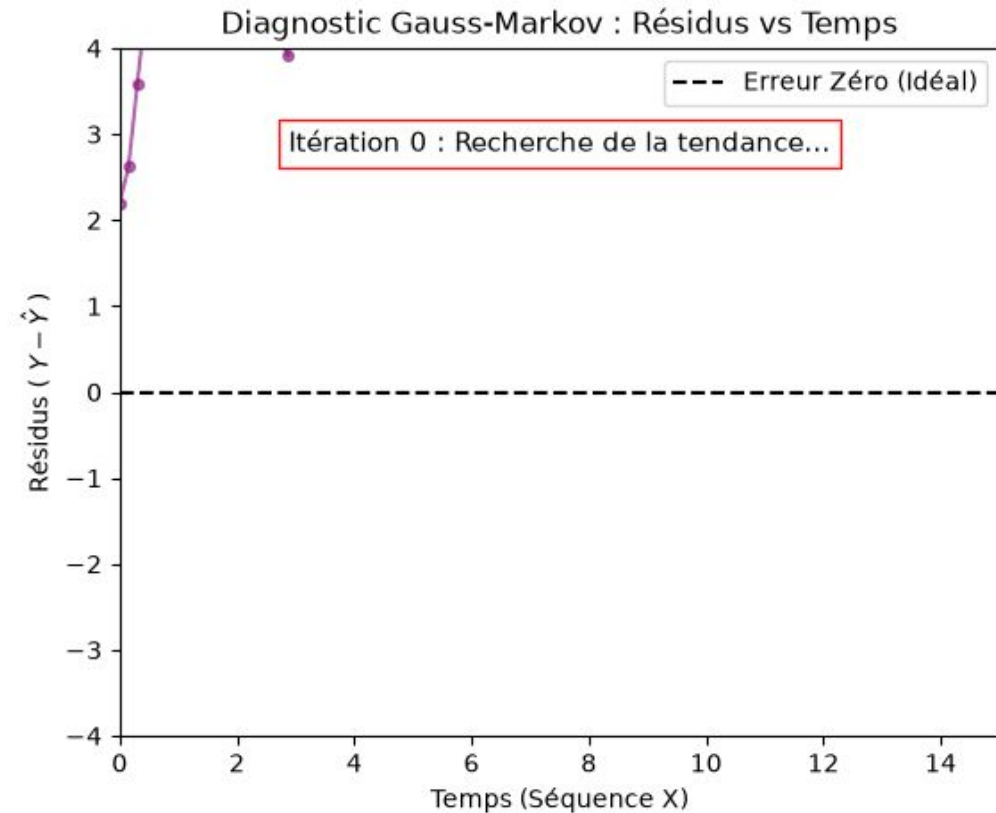
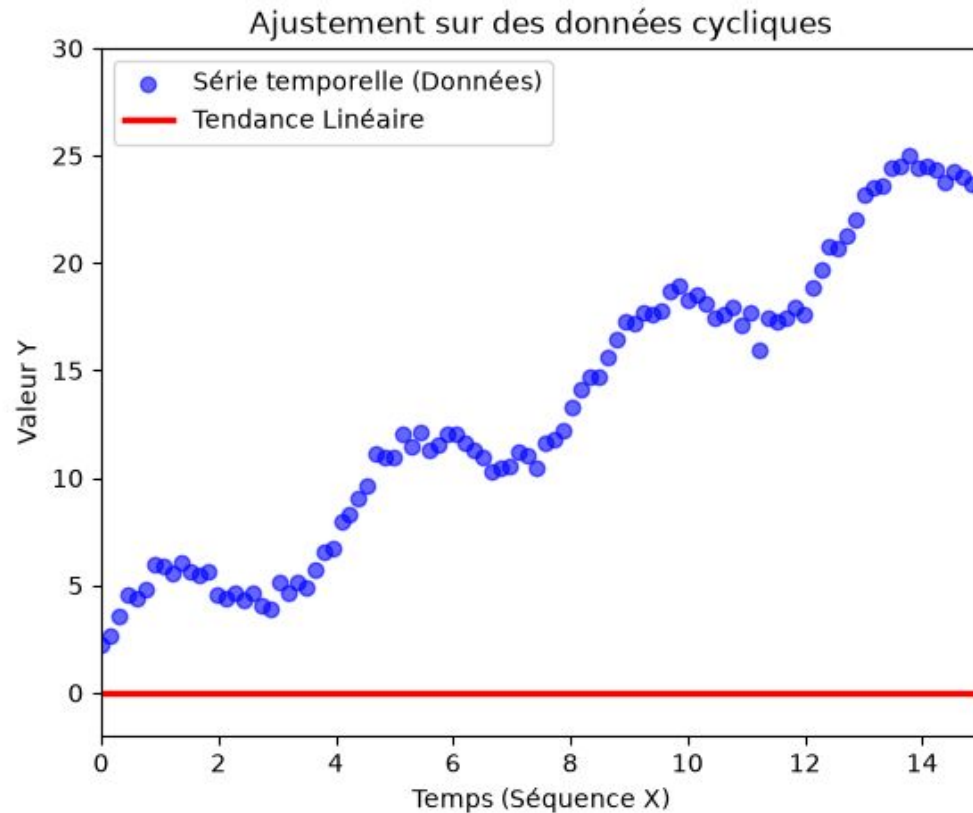


Cela rend les intervalles de confiance invalides, car la marge d'erreur n'est plus prévisible.

Condition 3 : Indépendance

Les erreurs (résidus) ne doivent présenter aucune corrélation entre elles.

Vérification de l'Indépendance (Gauss-Markov)

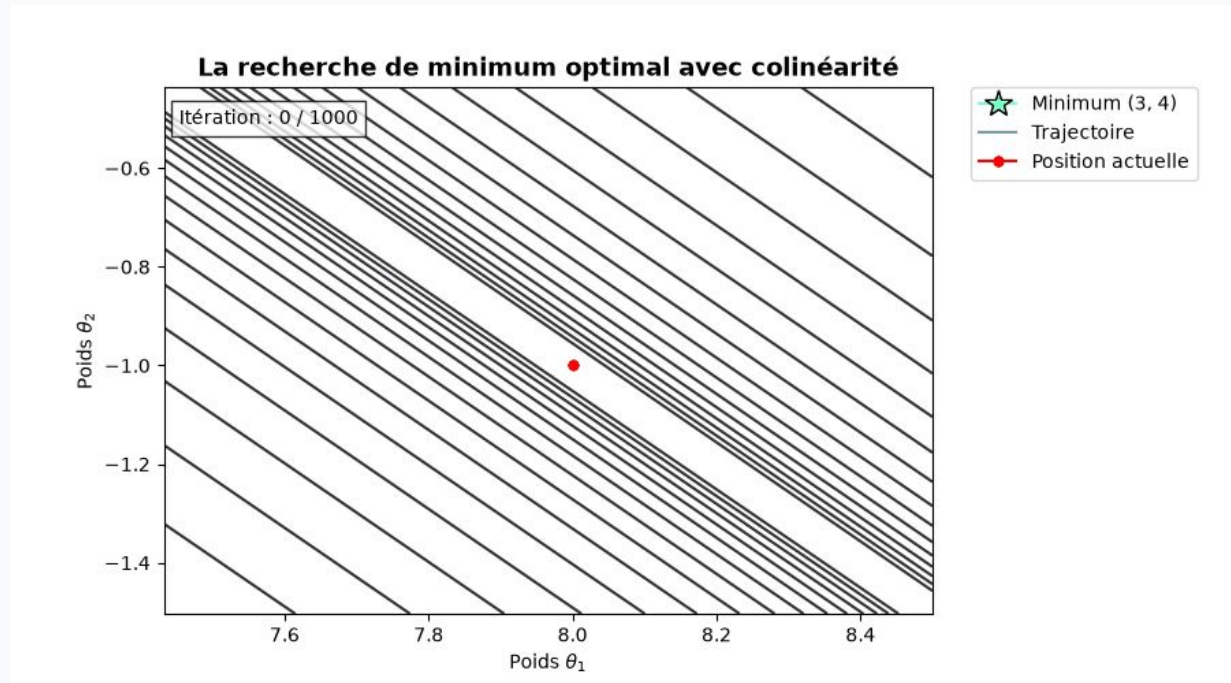


Le problème d'autocorrélation survient souvent dans les séries temporelles, où l'erreur d'hier influence l'erreur d'aujourd'hui.

Condition 4 : Non-colinéarité

La non-colinéarité est la condition qui stipule que vos variables explicatives ne doivent pas être fortement corrélées entre elles.

En présence de colinéarité, le modèle n'arrive pas à déterminer laquelle des deux variables a réellement un impact sur Y, car elles évoluent exactement de la même manière.



Généralisation du Modèle

**Train, Test et capacité prédictive sur de nouvelles
données.**

Le Danger de l'Entraînement Seul

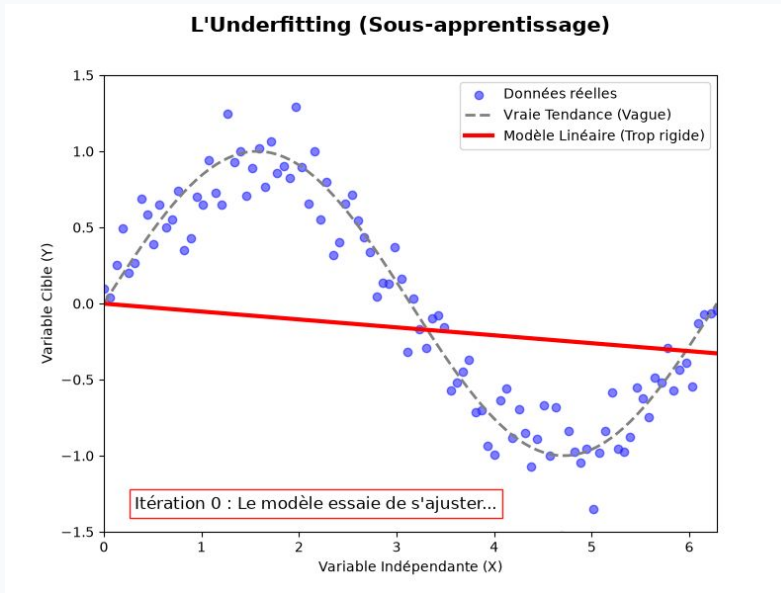
Mémoriser n'est pas apprendre

Une optimisation aveugle sur des données de Train (entraînement) ne vaut rien si le modèle mémorise le bruit au lieu de la tendance fondamentale.

La base de Test

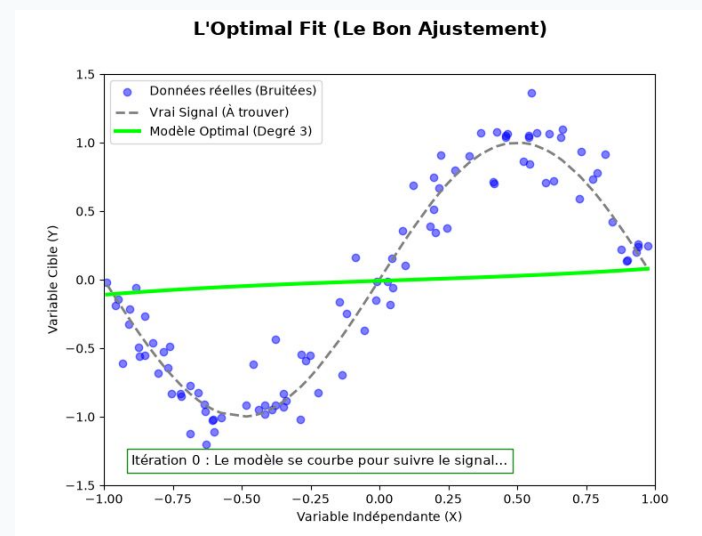
Il est impératif de séparer une portion de la base de données (Test Set) pour évaluer de manière indépendante les performances futures du modèle.

Problèmes de Convergence



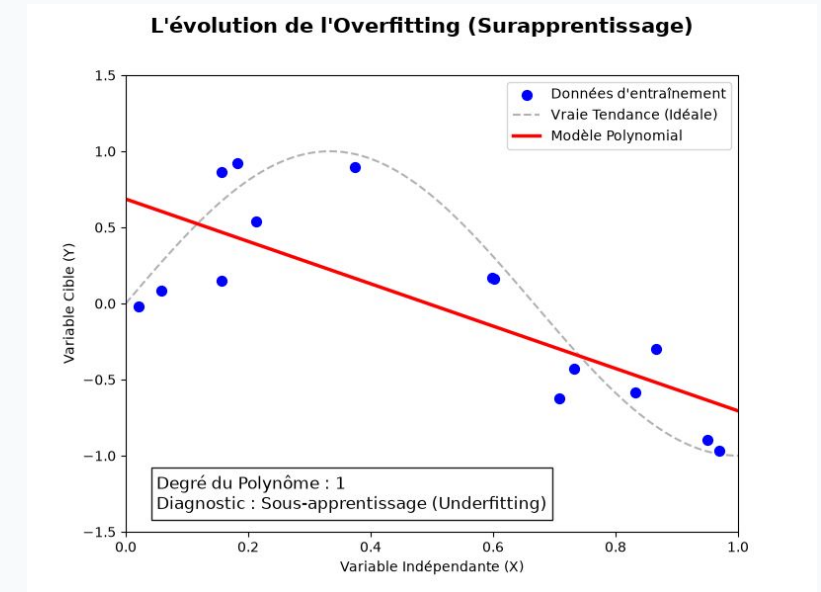
Underfitting

Sous-apprentissage. Modèle trop simple, ligne droite incapable de capter la courbe.



Ajustement Optimal

Le modèle saisit la tendance fondamentale sans mémoriser le bruit.



Overfitting

Sur-apprentissage. Le modèle relie chaque point (bruit) chaotiquement.

Les Outliers

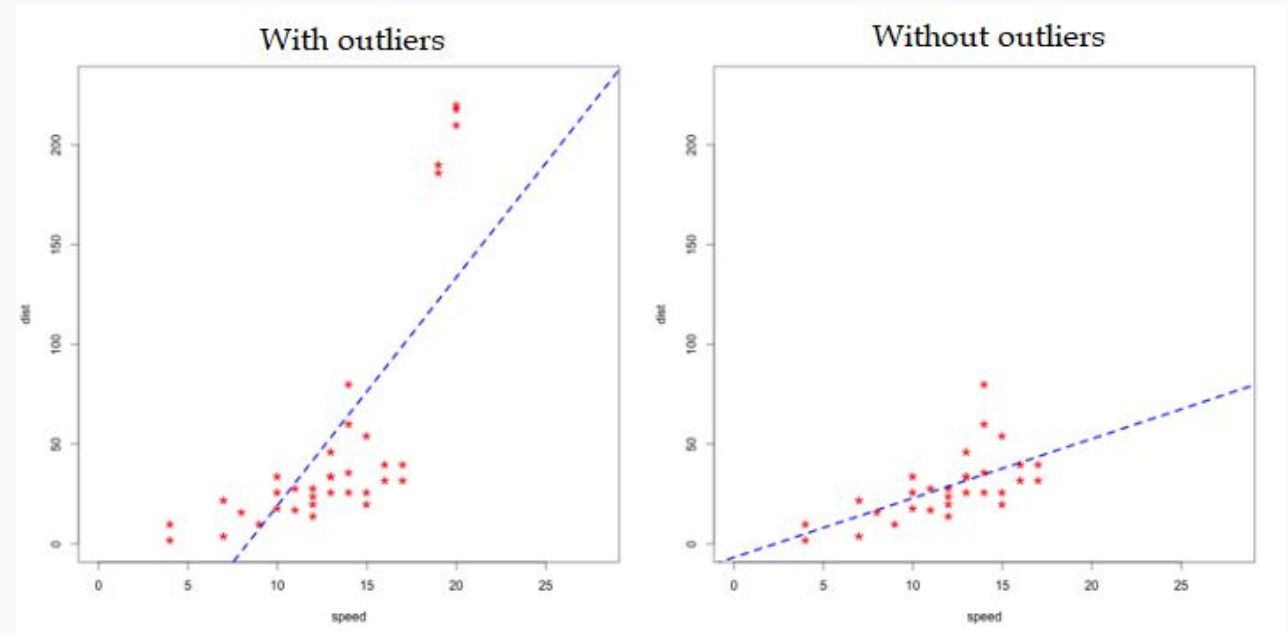
**L'impact destructeur des valeurs aberrantes sur la régression
linéaire.**

La menace des Outliers

L'aimant statistique

L'algorithme MCO minimise la distance des erreurs au carré. Une seule valeur aberrante pèse extrêmement lourd.

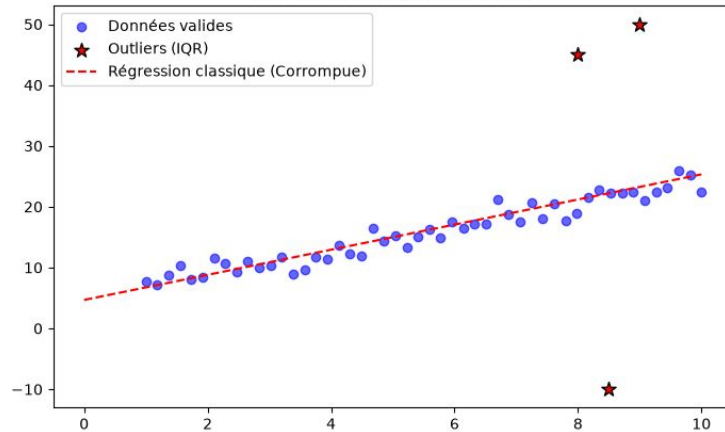
Cet outlier attire la droite de régression vers lui, détruisant la fiabilité des prédictions pour toutes les autres données normales.



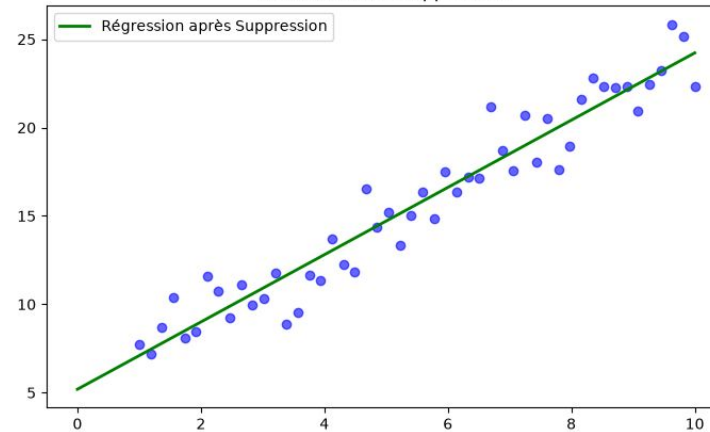
Défier la Régression

Gestion des Outliers : Détection, Traitement et Régression Robuste

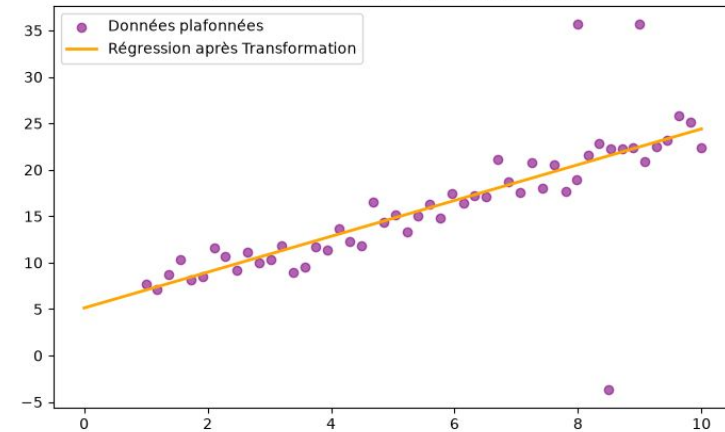
1. Identification (Méthode IQR)



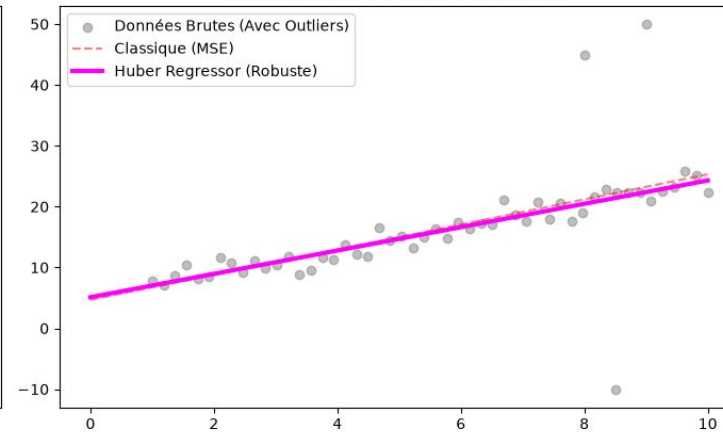
2. Traitement : Suppression



3. Traitement : Transformation (Plafonnement)



4. Modélisation Robuste (Huber Loss)



Traiter les anomalies

La présence d'outliers défie la validité de la régression linéaire.

Il faut les identifier (Z-score, IQR) et les traiter (suppression, transformation) ou utiliser des algorithmes plus robustes (ex: Huber Loss).

Interprétabilité des Modèles

**Comprendre et expliquer les décisions
algorithmiques.**

Boîte Blanche Vs Boîte Noire

Modèles Linéaires (Boîte Blanche)

Très facilement explicables. Chaque coefficient (Beta) indique explicitement l'impact d'une variable (ex: +1 chambre = +50k€).

Critique pour la conformité et la médecine.

Réseaux Profonds (Boîte Noire)

Les CNN ou LLM offrent des performances extrêmes mais sont opaques.

Il est quasi-impossible de tracer la logique exacte d'une décision isolée.

Régularisation

Pour combattre l'Overfitting dans la régression, on ajoute des pénalités :

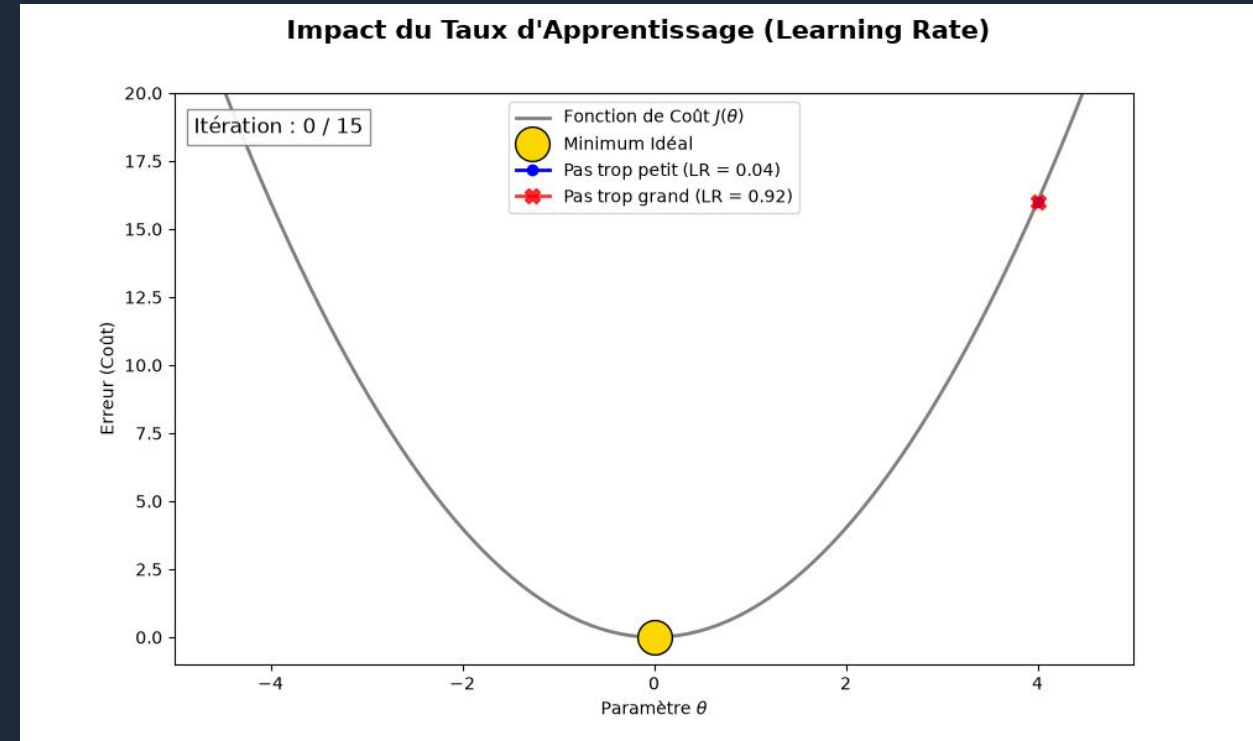
- **Régression Ridge (L2)** : Pénalise les grands coefficients pour lisser la courbe.
- **Régression Lasso (L1)** : Peut forcer certains coefficients à zéro, agissant comme sélectionneur de variables.

Taux d'Apprentissage (Alpha)

Dans la descente de gradient, l'Alpha contrôle la taille du "pas".

Trop grand : L'algorithme diverge et rate le minimum.

Trop petit : L'algorithme converge trop lentement (coût calculatoire).



Le Flux de Travail ML

De la donnée brute au modèle en
production.

Cycle de Vie



Préparation

Collecte, nettoyage, gestion des outliers et séparation Train/Test.



Modélisation

Choix de l'algorithme (Régression), Descente de Gradient et Régularisation.



Évaluation

Mesure de la précision (R^2), analyse des résidus et déploiement.

Outils Standards de l'Industrie



Python : Le langage dominant du Machine Learning.



NumPy & Pandas : Algèbre linéaire et manipulation massive des DataFrames.



Scikit-Learn : Implémentation clé en main des algorithmes (MCO, Gradient).

Conclusion Générale

**Des mathématiques fondamentales à l'intelligence artificielle
appliquée.**

Merci !

Questions ?